# Graph traversals and weighted graph algorithms

**William Hendrix**

*Lecture 10*

# Outline

- Review
  - Graph algorithm analysis
  - BFS
  - DFS
- Traversal complexity
- Traversal trees
- Applications of BFS/DFS
- Dijkstra's Algorithm
- Prim's Algorithm
- Kruskal's Algorithm
- Topological sorting

# Graph algorithm analysis review

- May include graph features
  - $n$: # vertices
  - $m$: # edges
  - deg($v$): degree of $v$
- Possibly consider graph operations separately

| Operation | Adjacency matrix | Adjacency list | Edge list |
|---|---|---|---|
| Graph(n) | $\Theta(n^2)$ | $\Theta(n)$ | $\Theta(1)$ |
| AddEdge(u, v) | $\Theta(1)$ | $\Theta(1)$* | $\Theta(m)$ |
| RemoveEdge(u, v) | $\Theta(1)$ | $\Theta(1)$* | $\Theta(m)$ |
| IsAdjacent(u, v) | $\Theta(1)$ | $\Theta(1)$* | $\Theta(\lg(m))$ |
| GetNeighbors(v) | $\Theta(n)$ | $\Theta(\deg v)$ | $\Theta(\mathrm{m})$ |
| **Space:** | $\Theta(n^2)$ | $\Theta(m+n)$ | $\Theta(m)$ |

- Handshaking Lemma: $\sum_{v \in V} \deg v = 2m = \Theta(m)$
  - All neighbors of all vertices = all edges

# Traversal review

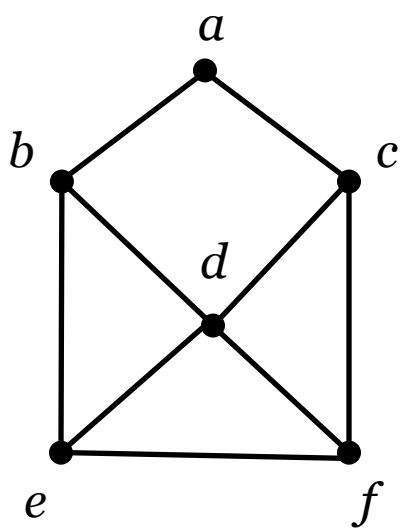
- BFS: queue-based traversal
  - Iteratively dequeue and add undiscovered neighbors
  - Discover when adding to queue, explore when removing
  - Outer loop finds next undiscovered node and restarts inner loop

- DFS: recursive (stack-based) traversal
  - Recursively explore each undiscovered neighbor
  - Discover before processing neighbors, explore afterwards
  - Outer function finds next undiscovered node and runs recursion

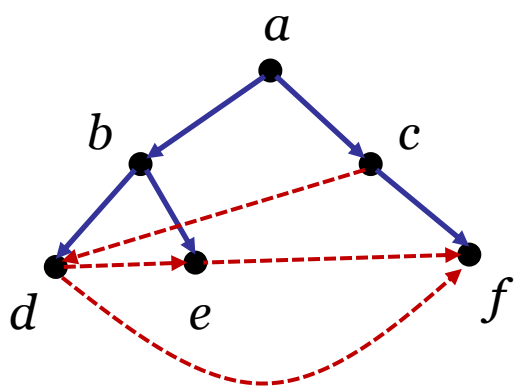
- Both explore vertices via edges
- Both mark vertices undiscovered, discovered, then explored
- Both are $\Theta(n + m)$ with adjacency list
  - Mark all vertices: $\Theta(n)$
  - Call getNeighbors on every node: $\Sigma \deg(v) = \Theta(m)$

# BFS and DFS traversal trees

- Traversal tree
  - Set of edges traversed by BFS or DFS,
  - Augmented with other edges of graph

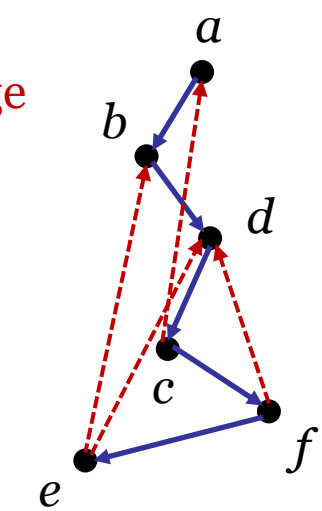



**BFS**




Tree edge
Non-tree edge

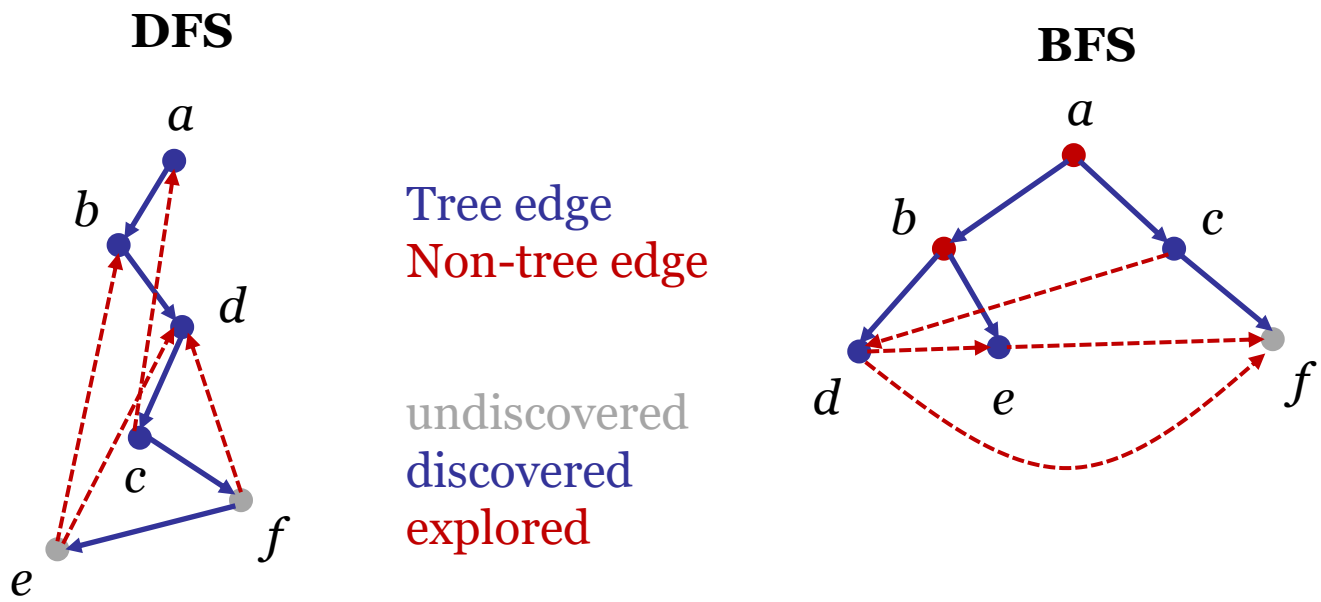
**DFS**




- Non-tree edges are siblings, cousins, or nieces/nephews
  - "Cross-edges"

- Non-tree edges are ancestors
  - "Back-edges"

# Traversal tree structure



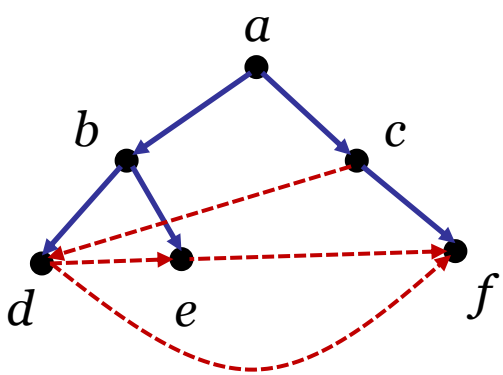

- **Observation 1:** non-tree edges connect to discovered vertices
  - Tree edges are undiscovered
- **Observation 2:** vertices are discovered iff they are in stack/queue
  - Stack: all vertices back to root
  - Queue: remaining vertices on same level + vertices on next level
- DFS: non-tree-edges are ancestors ("back edges")
- BFS: non-tree-edges are on same or next level ("cross edges")

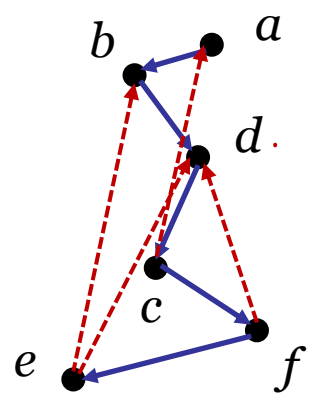
# Graph traversal example

- *Recall:* length of the *shortest* path between two vertices
  - In undirected graph, fewest number of edges

1. Should we use BFS or DFS? Why?

- *Hint:* what is *d(a, e)* in the graph below?




2. Prove that *d(a, v)* equals the level of *v* in a BFS traversal tree starting at *a*, for every vertex *v*.

# Traversal exercise

- Design an algorithm to label each vertex with a *component ID* such that all vertices in the same connected component have the same ID.
  - What is the runtime of your algorithm when using an adjacency list?

```
BFS:
Mark v_0 discovered
Enqueue v_0
While queue not empty:
    Dequeue vertex v
    Mark v explored
    Mark children discovered
    Enqueue children
If any vertex still undiscovered:
    Choose one and repeat
```

```
DFS:
While some vertices undiscovered:
    v = next undiscovered vertex
    DFS-Rec(v)

DFS-Rec(v):
Mark v as discovered
For undiscovered neighbors u:
    DFS-Rec(u)
Mark v explored
```

# BFS vs. DFS

Both BFS and DFS:

- Traverse one component at a time
- Take $\Theta(n+m)$ time
- Traversal trees have special properties
  - Sometimes critical, sometimes unimportant

- DFS is somewhat easier to implement
- Usage depends on application

# BFS and DFS application examples

**Both:**

- Component detection
- Cycle detection

**BFS:**

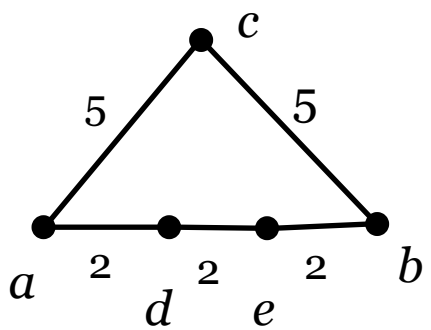
- Unweighted distance
- Radius and center of unweighted graph
- Betweenness centrality
  - Ranks vertices by “importance”
  - Counts shortest paths that pass through a vertex

**DFS:**

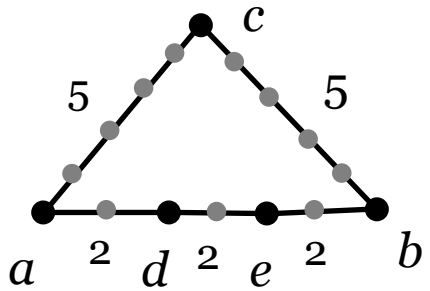
- Cut edge or cut vertex detection
- Sorting (on BST)
- Topological sorting
  - Order vertices of Directed Acyclic Graph (DAG) so all edges point to right
- *Backtracking*

# Weighted graph distance

- Edge weights may represent distance or strength of connection
  - E.g., road network or chemical interactions
- Distance: sum of edge weights
- BFS doesn't necessarily find shortest path:



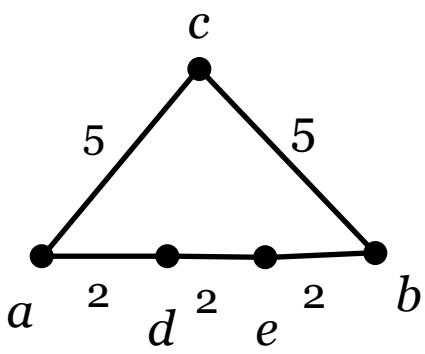

  - Finds path with fewest edges
- Naïve strategy: subdivide edges so that all edges have same length




- Finds shortest path
- Complexity depends on edge weights and GCD

# A better solution

- Don't actually add "interstitial" vertices
- Process vertices in same order as naïve strategy
  - Store when we would reach each vertex



# Dijkstra's algorithm

- Pseudocode

```
Input: G = (V, E): graph in question
Input: a, b: vertices to measure distance between
Input: n, m: order and size of G
Label all vertices as unreachable except a
v = a, D = 0
while v ≠ b do
    for u ∈ N(v) do
        Label u with distance D + d(v, u) unless u has a smaller label
    end
    Choose reachable vertex v with min distance D
end
return D
```

  - To return the path, add a back-link every time you label *u*
  - Traverse back-links from *b* to *a*

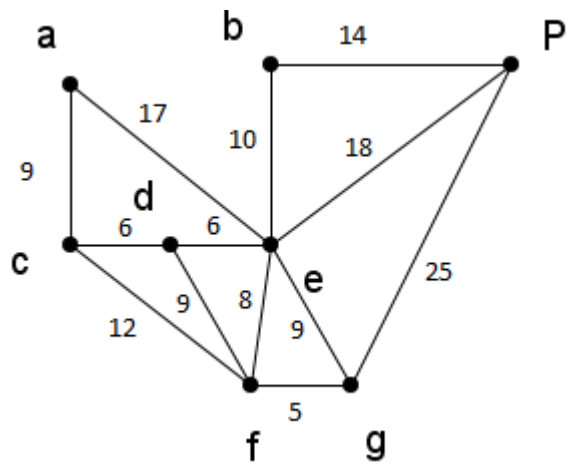
- Greedy algorithm
  - Always selects vertex with min distance

# Dijkstra's analysis

```
Input: G = (V, E): graph in question
Input: a, b: vertices to measure distance between
Input: n, m: order and size of G
Label all vertices as unreachable except a
v = a, D = 0
while v ≠ b do
    for u ∈ N(v) do
        Label u with distance D + d(v, u) unless u has a smaller label
    end
    Choose reachable vertex v with min distance D
end
return D
```

# Minimum spanning tree

- Consider the following scenario:
  - Imagine you are a creating a new power company
  - Need to connect customers to your power plant
  - Need to minimize costs
- **Example**
  - Connect *P* to *a*-*g*:



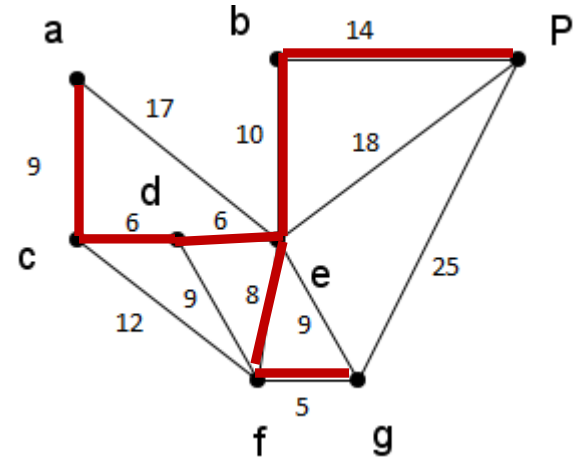

- Subgraph of a connected weighted graph that:
  - Connects all vertices of the graph (*spans*)
  - Has no cycles (*tree*)
  - Has the lowest edge weight sum among all spanning trees

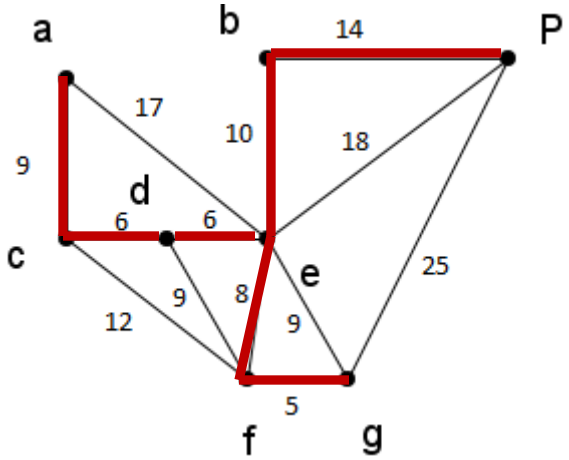
# Computing the MST

- Two main algorithms
  - Both greedy

- Prim's algorithm
  - Starts at one vertex
  - Adds edge to nearest unconnected vertex
  - Repeat until spanning

- Kruskal's algorithm
  - Sort edges
  - Adds min edge between disconnected vertices
  - Repeat until connected



# Prim's analysis

```
Input: G = (V, E): graph in which to find MST
Input: n, m: order and size of G
Output: Minimum spanning tree of G
edge = Array(n)
Let heap be a heap where every vertex has value ∞
Choose v_0 and decrease its value in heap to 0
for i = 1 to n do
    v = heap.DeleteMin()
    If v ≠ v_0, add (v, edge[v]) to MST
    for u ∈ N(v) do
        if u ∈ heap and d(u, v) < value(u) then
            Decrease value of u to d(u, v) in heap
            edge[u] = v
        end
    end
end
return MST
```

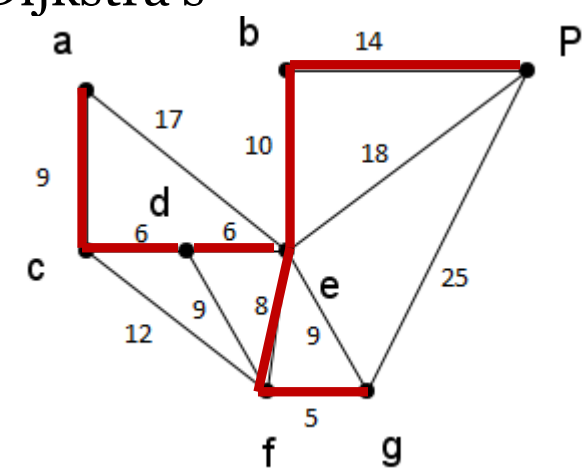
# Kruskal's analysis

```
Input: G = (V, E): graph in which to find MST
Input: n, m: order and size of G
Output: Minimum Spanning Tree of G
E' = sort(E)
uf = UnionFind(n)
for i = 1 to m do
    (u, v) = E'[i]
    if uf.Find(u) ≠ uf.Find(v) then
        Add (u, v) to MST
        uf.Union(u, v)
    end
end
return MST
```

# MST analysis

- Prim's vs. Kruskal's
  - Prim's: $\Theta(n + m \lg n)$
  - Kruskal's: $\Theta(n + m \lg m)$
  - Asymptotically equivalent
    - Prim's typically faster for dense graphs
    - Similar time for sparse graphs
  - Linear space for both

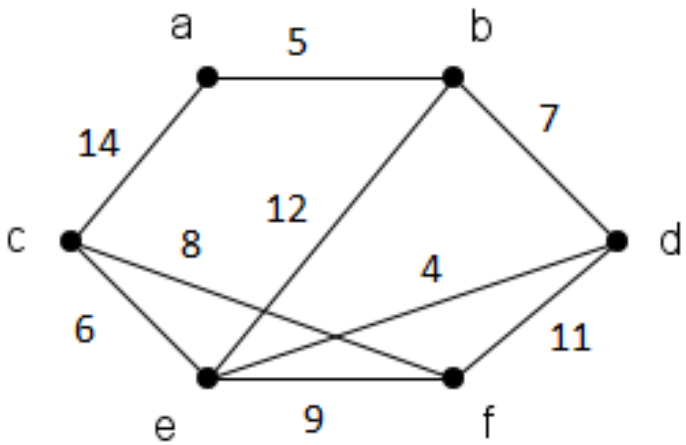
- Note: Prim's algorithm very similar to Dijkstra's
- BUT: MST ≠ min distance

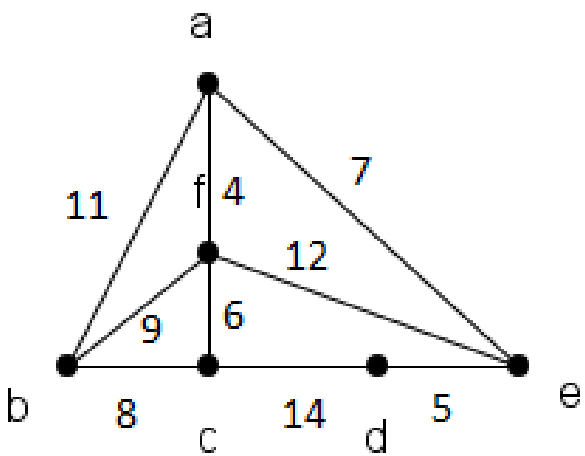


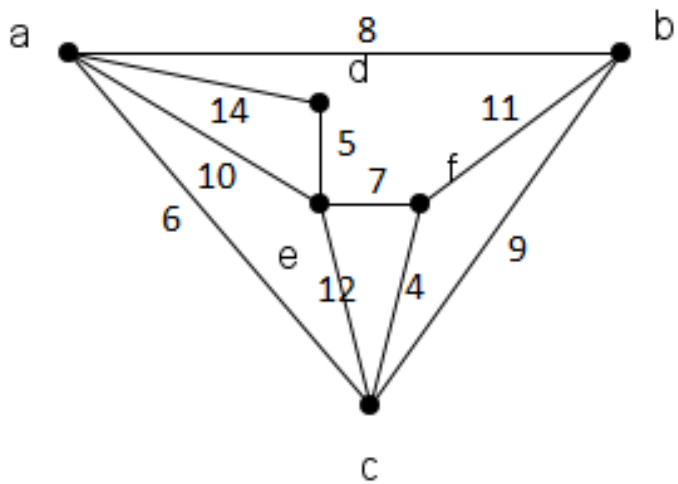

- MST Applications
  - Connecting spatial points
  - Travelling Salesman Problem
  - Clustering

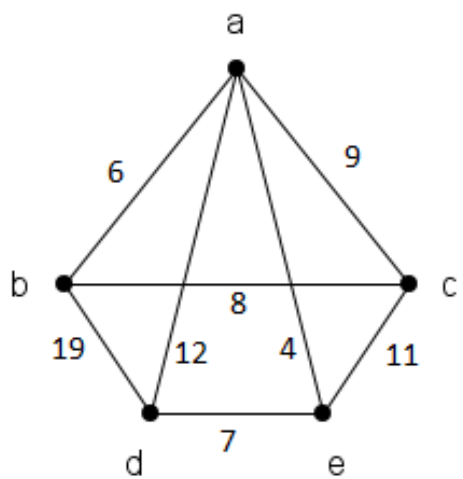
# Graph algorithm exercise

- For each graph below, compute the tree produced by Prim's and Dijkstra's algorithms, starting at *a*



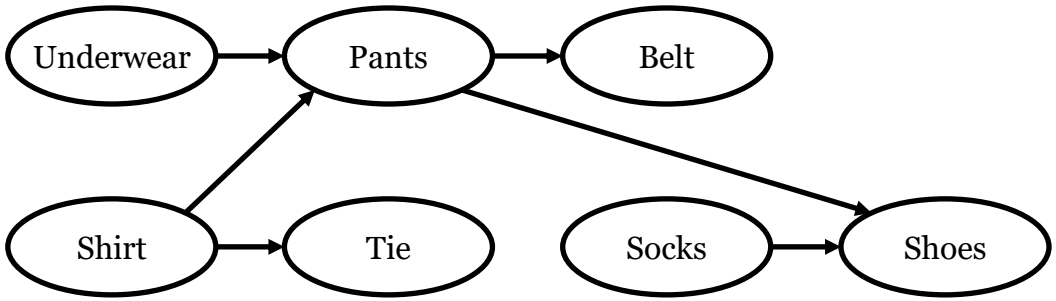



# Directed acyclic graphs

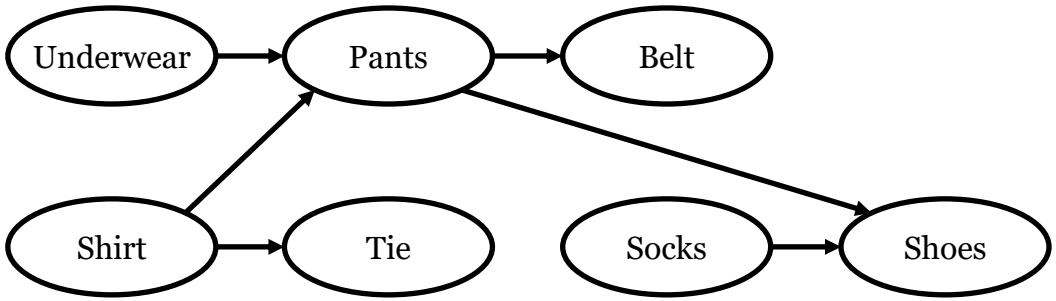
- Directed graphs with no cycles
- Sometimes used to represent processes or requirements
- **Example**




- *Important question*
  - How can I finish everything while meeting all requirements?

# Topological sorting

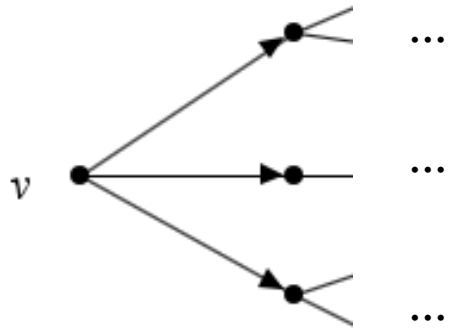
- **Input:** directed acyclic graph (DAG)
- **Output:** vertices ordered so that all edges point right
- **Example**




- **Algorithm**
  - Perform DFS
  - Add all vertices to stack when you mark them as explored
  - Return vertices in reverse order
- **Example (above)**
  - DFS: belt, shoes, pants, underwear, tie, shirt, socks
  - Reversed: socks, shirt, tie, underwear, pants, shoes, belt

# Topological sorting correctness

- *Proof sketch*
  - Everything that depends on $v$ is explored before $v$ is finished

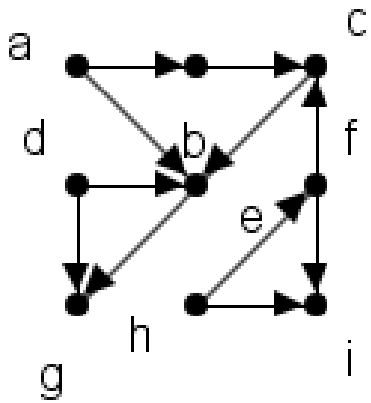



  - In reverse, $v$ appears before everything that depends on it

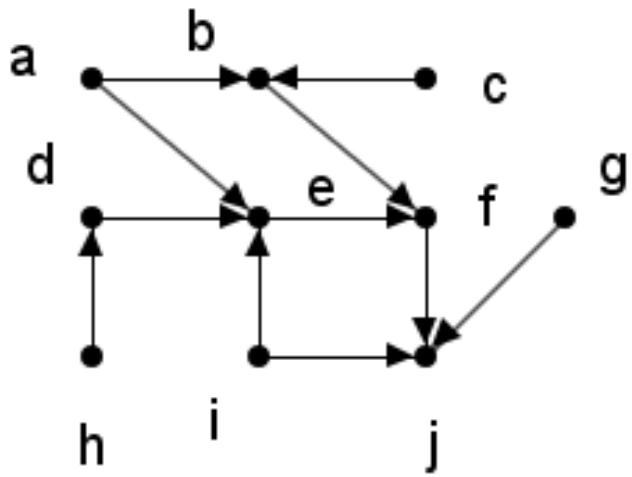
# Topological sorting exercise

Topologically sort the following DAGs:

1.




2.



# Coming up

- Weighted graph algorithms
- Backtracking

- **Recommended readings:** Sections 13.1-13.3, 14.1-14.2, and 15.1-15.3
- *Practice problems:* R-13.1, R-13.2, R-13.5, R-13.7, C-13.4, R-14.2, R-14.3, C-14.3, C-14.4, A-14.6, R-15.9, C-15.1, C-15.4, A-15.6